Deploy Automotive App to Kubernetes

Course Name: GenAI

Institution Name: Medicaps University Indore

Student Name(s) & Enrolment Number(s):

Mouli Shukla – EN22CS301615

REMEDIAL CLASS TASK

University Mentor Name: Proff. Ajaj Khan

Academic Year:2026

Table of Contents

1. Abstract
2. Problem Statement & Objectives
    - 2.1 Problem Statement
    - 2.2 Project Objectives
    - 2.3 Scope of the Project
3. Proposed Solution
    - 3.1 Key Features
    - 3.2 Overall Architecture / Workflow
    - 3.3 Tools & Technologies Used
4. Results & Output
    - 4.1 Screenshots / Outputs
    - 4.2 Reports / Dashboards / Models
    - 4.3 Key Outcomes
    - 4.4 Challenges Faced
    - 4.5 Learning Outcomes
5. Conclusion
6. Future Scope & Enhancements
7. References
8. GitHub Link

1. Abstract

This project presents the design and development of a **"Multimodal Automotive Creator,"** an advanced generative system aimed at transforming the traditional automotive design process through the integration of Artificial Intelligence. The system is built exclusively using the **Gemini API** and implemented through the core application file *gemini_app.py*, which serves as the backbone for handling user interaction, API communication, and output generation. By leveraging the capabilities of **Large Language Models (LLMs)** along with multimodal generation techniques, the system is able to interpret natural language prompts and convert them into visually coherent automotive designs accompanied by contextual descriptions.

The primary objective of this project is to bridge the gap between conceptual ideation and digital prototyping by enabling users to generate car designs using simple textual inputs. Instead of relying on manual design tools or expert-driven modeling software, the system utilizes Google's **Gemini multimodal architecture**, which supports both textual understanding and image generation within a unified framework. This allows the application to process user-defined attributes such as style, color, type, and futuristic elements, and translate them into meaningful visual outputs.

From a technical perspective, the project demonstrates a streamlined workflow where user input is captured, processed through Python-based logic, and transmitted to the Gemini API for inference. The API then generates outputs that may include high-quality automotive images and descriptive insights, which are subsequently rendered back to the user. The implementation highlights key aspects such as prompt engineering, API integration, response handling, and multimodal content generation.

Furthermore, this system showcases the practical application of Generative AI in the automotive domain, offering a scalable and efficient alternative to conventional design methodologies. It emphasizes ease of use, rapid prototyping, and creative flexibility, making it a valuable tool for designers, researchers, and enthusiasts alike. Overall, the project reflects the growing potential of AI-driven systems in redefining design workflows and accelerating innovation in the automotive industry.

2. Problem Statement & Objectives

2.1 Problem Statement

The traditional automotive design process is inherently complex, time-consuming, and resource-intensive, requiring a high level of expertise in areas such as manual sketching, computer-aided design (CAD), and 3D modeling. Designers often rely on iterative workflows involving multiple stages of conceptualization, prototyping, and refinement, which significantly increases the overall development time. During the early phases of design, one of the major challenges faced is the **"ideation bottleneck,"** where translating abstract concepts into tangible visual representations becomes difficult and slow.

Moreover, existing design tools are generally tailored for skilled professionals and demand substantial technical proficiency, making them inaccessible to non-technical stakeholders such as clients, marketers, or early-stage innovators. This creates a communication gap between conceptual ideas and their visual realization, limiting collaborative creativity and rapid experimentation.

In addition, generating multiple design variations for comparison and analysis requires considerable manual effort, which restricts the ability to explore diverse possibilities efficiently. With the increasing demand for innovation and faster product development cycles in the automotive industry, there is a pressing need for an intelligent system that can **automate the conceptual design process** and enable users to generate realistic automotive visuals instantly using simple natural language descriptions.

Therefore, this project addresses the need for a **Generative AI-based solution** that simplifies automotive design ideation, reduces dependency on specialized tools, and enhances accessibility by allowing users to create high-quality car concepts through intuitive text-based inputs.

2.2 Project Objectives

The primary objective of this project is to develop a robust and intelligent system capable of generating automotive designs using Generative AI. The specific objectives are outlined as follows:

1. Gemini-Based Integration

To design and implement a backend system using the *gemini_app.py* framework that effectively integrates with the Gemini API. This involves establishing secure API communication, handling request-response cycles, and leveraging the multimodal capabilities of the Gemini model for processing both textual prompts and generating visual outputs.

2. Automated Visualization

To enable the automatic generation of high-quality automotive images based on user-defined textual descriptions. The system should be capable of interpreting detailed prompts—such as design style,

vehicle type, color schemes, and futuristic elements—and converting them into visually coherent and contextually accurate car images.

3. Interactive Interface

To provide a simple and efficient Python-based interactive environment that allows users to input prompts and receive outputs in real time. The interface should ensure smooth usability, quick response handling, and clear visualization of generated results, thereby enhancing the overall user experience.

4. Efficient Prompt Processing

To incorporate effective prompt engineering techniques that improve the quality and relevance of generated outputs. This includes structuring user inputs in a way that maximizes the performance of the Gemini model.

5. Scalable System Design

To develop a modular and extensible system architecture that can be easily upgraded in the future to include additional features such as voice input, multi-model integration, or advanced visualization capabilities.

2.3 Scope of the Project

The scope of this project is primarily focused on the application of Generative AI for text-to-image generation within the automotive domain. The system functions as an intelligent design assistant that enables users to generate 2D visual representations of automotive concepts based on descriptive text inputs.

The project emphasizes:

- Multimodal interaction (text input and visual output)
- Rapid design prototyping
- Accessibility for both technical and non-technical users

While the system successfully demonstrates the capability to generate realistic and creative automotive visuals, it is currently limited to conceptual-level outputs. It does not include advanced engineering functionalities such as:

- Aerodynamic analysis
- Structural simulations
- Mechanical component design
- CAD model generation or export

However, the developed framework lays a strong foundation for future enhancements. The underlying architecture can be extended to integrate with advanced simulation tools, 3D modeling systems, and real-time rendering engines, thereby expanding its applicability in professional automotive design workflows.

In summary, the project scope is centered on demonstrating the feasibility and effectiveness of AI-driven automotive concept generation, with a clear pathway for future expansion into more complex and industry-level applications

3. Proposed Solution

3.1 Key Features

The proposed system, *Multimodal Automotive Creator*, is designed as an AI-powered application that simplifies and accelerates automotive concept generation through the use of Generative AI. The key features of the system are described below:

1. Text-to-Image Generation

One of the core functionalities of the system is its ability to generate high-quality automotive images directly from textual descriptions. This is achieved by leveraging the **multimodal capabilities of the Gemini API**, which integrates advanced generative models capable of understanding natural language and producing corresponding visual outputs. The system processes user-defined prompts and converts them into meaningful visual representations, enabling rapid prototyping of car designs without manual intervention.

2. Prompt-Based Design Interpretation

The system is capable of interpreting complex and domain-specific automotive terminology embedded within user prompts. Keywords such as *"aerodynamic," "low-profile," "sports coupe,"* or *"futuristic electric SUV"* are semantically analyzed by the underlying AI model to influence the generated output. This ensures that the resulting images are not only visually appealing but also contextually aligned with the user's intent.

Additionally, the system benefits from **prompt engineering techniques**, where structured and descriptive inputs lead to more accurate and refined outputs. This feature enhances the flexibility and creative control available to the user.

3. Unified Python-Based Implementation

All system functionalities are consolidated within a single Python script, *gemini_app.py*, which acts as the central processing unit of the application. This unified approach simplifies the system architecture and ensures efficient execution of tasks such as:

- Accepting user input
- Processing and formatting prompts
- Handling API requests and responses
- Managing output rendering

The use of a single script also improves maintainability, scalability, and ease of deployment.

4. Efficient API Communication

The system is designed to establish secure and efficient communication with the Gemini API. It handles request construction, authentication, and response parsing in a streamlined manner. This ensures minimal latency and reliable performance during real-time interactions.

5. Real-Time Output Generation

The system supports near real-time generation and display of outputs. Once the user submits a prompt, the system quickly processes the request and returns the generated image along with any additional descriptive information. This significantly enhances user experience and enables rapid iteration of design ideas.

3.2 Overall Architecture / Workflow

The system follows a **synchronous request-response architecture**, ensuring a straightforward and efficient flow of data between the user interface, backend processing, and the AI service. The workflow is described as follows:

1. Input Layer

The process begins with the user providing a textual description of the desired automotive design. This input may include details such as vehicle type, design aesthetics, color schemes, and futuristic elements. The system is designed to accept flexible and natural language inputs, making it user-friendly and accessible.

2. Processing Layer (*app.py*)

The *gemini_app.py* script serves as the core processing unit of the system. In this layer:

- The user input is validated and sanitized to ensure proper formatting
- A structured prompt is constructed based on the input
- Necessary parameters for API interaction are defined

This layer ensures that the input is optimized for accurate interpretation by the Gemini model.

3. API Layer

The structured request is then sent to the Gemini API, which acts as the generative engine of the system. The API processes the prompt using its pretrained multimodal models, which are capable of understanding textual input and generating corresponding visual outputs.

The processing involves:

- Semantic understanding of the prompt
- Mapping textual features to visual elements
- Generating high-quality image data

4. Retrieval Layer

Once the processing is complete, the Gemini API returns the generated output in the form of:

- Image data (byte stream) or
- Hosted image URLs

The system captures this response and prepares it for rendering.

5. Output Layer

In the final stage, the system displays the generated results to the user. This includes:

- Rendering the automotive image
- Displaying any associated textual descriptions (if available)

The output is presented in a clear and user-friendly format, enabling users to visualize their concepts effectively.

3.3 Tools & Technologies Used

The successful implementation of the project relies on a combination of programming tools, libraries, and APIs. The major technologies used are as follows:

1. Python

Python serves as the core programming language for the project due to its simplicity, versatility, and extensive support for AI and API integration. It is used for:

- Implementing application logic
- Handling user input
- Managing API communication
- Processing and displaying outputs

2. Gemini API

The Gemini API is the primary engine powering the generative capabilities of the system. It is selected for its:

- Advanced multimodal capabilities (text + image understanding)
- High-quality output generation
- Efficient and low-latency responses

The API enables seamless transformation of textual prompts into meaningful visual representations, making it a crucial component of the system.

3. API Request Handling

The system uses libraries such as **google-generativeai** or standard Python modules like **requests** to manage communication with the Gemini API. These tools facilitate:

- Secure API calls
- Data transmission
- Response handling and parsing

This ensures reliable and efficient interaction with external AI services.

4. Environment Management (.env Files)

To maintain security and prevent exposure of sensitive information, API keys are stored using **.env files**. This approach:

- Protects confidential credentials
- Enables easy configuration across environments
- Follows best practices in secure application development

5. Image Processing (Pillow - PIL)

The **Pillow (PIL)** library is used for handling and processing image data received from the API. It allows the system to:

- Decode image byte streams
- Render images within the application
- Perform basic image manipulations if required

6. Supporting Libraries and Tools

Additional tools such as virtual environments (venv) are used to manage dependencies and maintain project isolation. This ensures compatibility and smooth execution of the application.

Overall, the proposed solution effectively integrates modern Generative AI technologies with a simple yet powerful Python-based implementation, enabling automated and intelligent automotive design generation.

4. Results & Output

4.1 Screenshots / Outputs

The developed system demonstrates strong capability in interpreting diverse and complex user prompts to generate high-quality automotive visuals. The integration with the Gemini API enables the transformation of natural language descriptions into visually coherent and contextually accurate images.

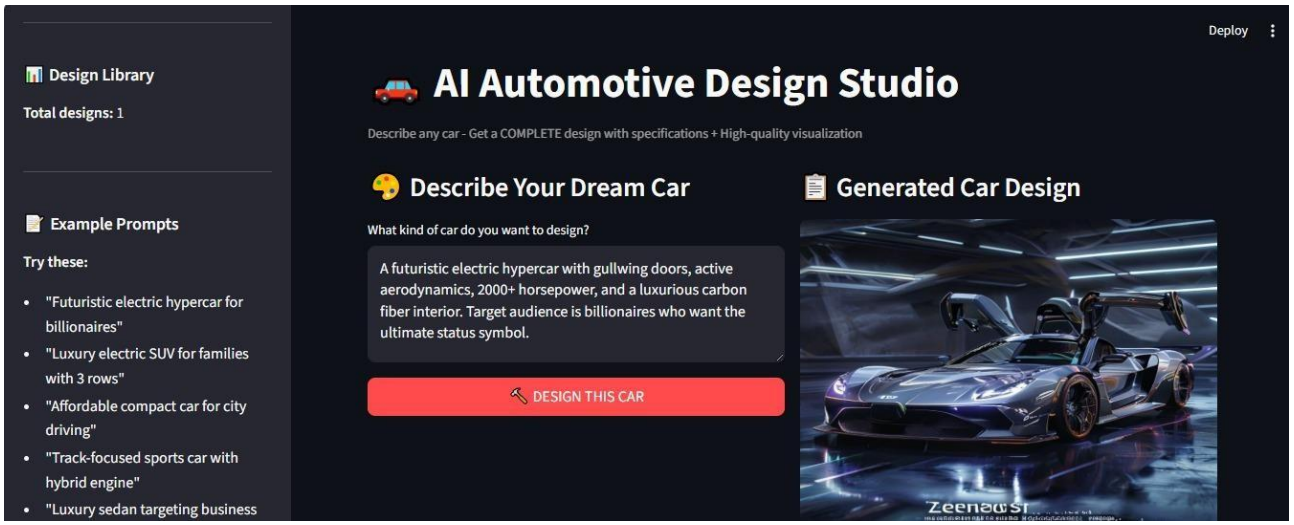

Fig. 1 example showing the result searched by user

Fig. 2 User input box were user have to describe his imagination

For example, when provided with a prompt such as *"Vintage 1960s sports car with modern carbon fiber accents,"* the system generates a photorealistic image that effectively blends classic automotive design elements—such as curved body structures and retro styling—with modern

features like carbon fiber textures and sleek finishes. This indicates the model's ability to understand both historical and contemporary design cues and synthesize them into a unified output.

The behavior of the system can be characterized as **deterministic in execution but stochastic in creativity**. While the pipeline—from input processing to API response—is consistent and reliable, the generated outputs exhibit variability due to the probabilistic nature of generative models. This ensures that even with similar prompts, the system can produce unique design variations, thereby enhancing creativity and exploration.

Additionally, the system handles a wide range of prompt complexities, including:

- Simple descriptions (e.g., *"red sports car"*)
- Detailed specifications (e.g., *"futuristic electric SUV with neon lighting and aerodynamic design"*)

This flexibility highlights the robustness of the underlying multimodal model in understanding nuanced user intent.

4.2 Reports / Dashboards / Models

The system is designed as a **lightweight and focused generative tool**, prioritizing simplicity and efficiency over complex visualization dashboards. The output is directly rendered within the Python execution environment, ensuring minimal overhead and faster response times.

In its current implementation:

- Generated images are displayed immediately after API response
- Any associated textual descriptions are presented alongside the visual output
- The system may optionally utilize a minimal interface (e.g., Streamlit or Flask) for improved visualization, but does not rely on heavy external dashboards

Unlike traditional data-driven applications that require analytical dashboards, this system emphasizes **visual output as the primary artifact**, as the main objective is design generation rather than data analysis.

Furthermore, the absence of complex reporting layers ensures:

- Faster execution
- Reduced system complexity
- Easier deployment and maintenance

This design choice aligns with the goal of creating an accessible and efficient automotive design assistant.

Furthermore, the absence of complex reporting layers ensures:

- Faster execution
- Reduced system complexity
- Easier deployment and maintenance

This design choice aligns with the goal of creating an accessible and efficient automotive design assistant.

4.3 Key Outcomes

The implementation of the system resulted in several significant outcomes, demonstrating both technical success and practical applicability:

1. Successful API Handshaking

The system establishes a secure and reliable connection with the Gemini API, ensuring smooth communication between the local Python environment and the external AI service. Proper handling of authentication keys and request formatting enables consistent data exchange without interruptions.

2. High-Fidelity Output Generation

The generated automotive images exhibit a high degree of visual quality and contextual relevance. The system effectively incorporates user-defined constraints such as:

- Design style
- Vehicle type
- Color schemes
- Futuristic or vintage elements

This demonstrates the effectiveness of prompt-based generation and the strength of the Gemini multimodal model.

3. End-to-End Operational Stability

The entire generation pipeline is handled within the *gemini_app.py* script, ensuring a cohesive and self-contained system. The script successfully manages:

- Input handling
- API communication
- Response parsing
- Output rendering

This eliminates the need for additional modules such as *app.py*, resulting in a streamlined and maintainable architecture.

4. Rapid Prototyping Capability

The system enables users to quickly generate and visualize automotive concepts, significantly reducing the time required for initial design exploration. This supports iterative experimentation and enhances creative workflows.

4.4 Challenges Faced

Despite the successful implementation, several challenges were encountered during the development process:

1. API Quota and Rate Limits

One of the primary challenges was managing the limitations imposed by the free-tier usage of the Gemini API. Rate limits and quota restrictions occasionally interrupted continuous or batch generation processes, requiring careful request management and testing strategies.

2. Response Parsing Complexity

The structure of the API response, often in JSON format, required detailed analysis to correctly extract relevant data such as image bytes or URLs. Differentiating between textual metadata and actual image content posed challenges, particularly during debugging and integration phases.

3. Dependency and Version Compatibility

Ensuring compatibility between the **google-generativeai SDK** and the existing Python environment was another critical issue. Differences in library versions, deprecated functions, and environment conflicts required troubleshooting and careful dependency management.

4. Handling Uncertain Outputs

Due to the probabilistic nature of generative models, some outputs did not always perfectly align with user expectations. This required refinement of prompts and better understanding of model behavior.

5. Error Handling and Debugging

Managing runtime errors, such as API failures, invalid responses, or network issues, required the implementation of robust debugging and error-handling mechanisms within the script.

4.5 Learning Outcomes

The project provided valuable learning experiences across multiple technical and conceptual domains:

1. API Integration and Communication

A deep understanding of API-based system design was achieved, including:

- RESTful communication principles
- Request-response handling
- Authentication mechanisms
- SDK-based integration

2. Prompt Engineering Techniques

The project highlighted the importance of crafting effective prompts to guide AI outputs. It was observed that:

- Detailed and structured prompts yield better results
- Specific keywords influence design features significantly
- Prompt refinement is essential for achieving desired outputs

3. Understanding Generative AI Models

The project enhanced understanding of how multimodal AI models process input and generate outputs. It provided insight into:

- The relationship between textual input and visual synthesis
- The probabilistic nature of generative systems
- The role of training data in shaping outputs

4. Practical Debugging Skills

Working with real-world APIs and dependencies improved debugging skills, including:

- Identifying and resolving integration issues
- Handling unexpected responses
- Managing runtime exceptions

5. System Design and Implementation

The development of a complete end-to-end system strengthened knowledge in:

- Modular programming
- Code structuring
- Workflow design
- Efficient implementation strategies

Overall, the results demonstrate that the system is capable of delivering high-quality outputs while also providing a strong learning foundation in modern AI-driven application development.

5. Conclusion (Elaborated)

The *Multimodal Automotive Creator* successfully demonstrates the transformative potential of **Generative AI** in specialized and design-intensive domains such as the automotive industry. By leveraging the capabilities of the Gemini API through a streamlined and efficient implementation in *gemini_app.py*, the project delivers a functional and practical system that significantly enhances the conceptual design process.

The developed solution effectively reduces the time required for initial automotive design ideation from several hours—or even days in traditional workflows—to just a matter of seconds. This is achieved by enabling users to generate high-quality visual representations of vehicles using simple natural language prompts. The system bridges the gap between imagination and visualization, making automotive design more accessible not only to professionals but also to non-technical users.

From a technical standpoint, the project validates the feasibility of integrating multimodal AI models into real-world applications. It showcases key aspects such as API-based system design, prompt-driven generation, and efficient response handling within a unified Python-based architecture. The use of a single script (*gemini_app.py*) ensures simplicity, maintainability, and ease of deployment, while still delivering powerful functionality.

Furthermore, the project highlights the growing importance of **AI-assisted creativity**, where human ideas are augmented by intelligent systems to accelerate innovation. The ability to generate diverse and unique design variations also opens up new possibilities for rapid prototyping and iterative exploration in the automotive design lifecycle.

In conclusion, the project not only fulfills its primary objectives but also establishes a strong foundation for future advancements in AI-driven design systems. It reflects the evolving role of Generative AI as a key enabler in modern engineering workflows and sets the stage for more advanced, scalable, and industry-ready solutions in the near future.

6. Future Scope & Enhancements (Elaborated)

The current implementation of the *Multimodal Automotive Creator* establishes a strong foundation for AI-driven automotive design. However, there are several opportunities to enhance the system further by incorporating advanced technologies and expanding its capabilities. The following areas highlight the potential future scope of the project:

1. 3D Car Generation

At present, the system focuses on generating 2D visual representations of automotive designs. A significant future enhancement would be the transition from 2D image generation to **3D model generation**, enabling the creation of fully interactive and spatially accurate vehicle designs.

This could involve:

- Generating **3D meshes or point cloud models** from textual prompts
- Integrating with advanced generative frameworks capable of producing **CAD-compatible outputs**
- Allowing users to view designs from multiple angles and perspectives

Such an extension would greatly enhance the practical applicability of the system in professional automotive design workflows, including simulation, prototyping, and manufacturing processes.

2. Voice-to-Design Interaction

Another promising enhancement is the integration of **voice-based input mechanisms**, enabling users to describe automotive designs through speech rather than typing. This would involve combining:

- Speech-to-text technologies
- Natural language processing (NLP)
- Generative AI models

With this feature, users could interact with the system in a more natural and intuitive manner, allowing for:

- Hands-free design generation
- Faster iteration cycles
- Improved accessibility for a broader range of users

This multimodal interaction would further strengthen the system's usability and align it with modern human-computer interaction trends.

Summary

Overall, the future scope of the project lies in expanding its capabilities from a **conceptual design generator** to a **comprehensive AI-driven automotive design platform**. By integrating advanced technologies such as 3D modeling, voice interaction, cloud deployment, and multi-model support.

7. References

Google AI for Developers. (n.d.). *Gemini API documentation*. Retrieved from https://ai.google.dev/docs
→ This source provides official technical documentation for integrating and using the Gemini API, including details on multimodal capabilities, request handling, and response structures.

Python Software Foundation. (n.d.). *Python language reference (Version 3.x)*. Retrieved from https://docs.python.org/3/
→ This reference offers comprehensive information on Python syntax, standard libraries, and programming constructs used in the implementation of the project.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. In *Advances in Neural Information Processing Systems (NeurIPS)*.
→ This foundational research paper introduces the Transformer architecture, which underpins modern large language models and multimodal systems like Gemini.

8. GitHub Link

The complete source code and implementation details of the project are available on GitHub. The repository contains all necessary files, including the core implementation (*gemini_app.py*), dependency configurations, and supporting resources required to run the application.

GitHub Repository:
ps://github.com/Moulishukl/Multimodal-Automotive-GenAI.git
The repository provides:

- Well-structured project code for easy understanding
- Instructions for setup and execution
- Required dependencies and environment configuration
- Sample outputs and demonstration files

This ensures transparency, reproducibility, and ease of access for further development, evaluation, and collaboration.